Modern NLP Interpretability

Introduction

Neural networks are no longer simple classifiers. Modern **large language models have hundreds of billions of parameters** and exhibit **emergent capabilities**.

Interpretability is the science of **reverse-engineering these systems to understand what they learn and how they compute**. It's essential for safety, debugging, and building trust in deployed systems.

Introduction

Before transformers, interpretability **focused on simpler questions**. We could **examine learned word embeddings, visualize attention in sequence-to-sequence models, and probe hidden representations** for linguistic properties.

These methods **assumed models were relatively shallow** and that their **internal representations were human-interpretable by design**. This assumption **breaks down completely for modern transformers**.

Introduction

Classical Method 1: Embedding Spaces

Word2vec and GloVe embeddings could be visualized in 2D using dimensionality reduction. We found that semantic relationships formed geometric patterns.

$$\vec{v}_{\text{king}} - \vec{v}_{\text{man}} + \vec{v}_{\text{woman}} \approx \vec{v}_{\text{queen}}$$

This worked because **embeddings were low-dimensional** (typically 300 dimensions) and **trained with explicit semantic objectives** like predicting context windows.

Introduction

Classical Method 1: Embedding Spaces

Word2vec optimizes: $\max_\theta \sum_{(w,c)\in D} \log \sigma(\vec{w} \cdot \vec{c})$

where w is a word, c is its context, and σ is the sigmoid function. **This objective encourages words with similar contexts to have similar vectors**.

For example, gender relationships are consistently present in contexts, so they become geometric directions in the embedding space.

Introduction

Classical Method 2: Attention Visualization

Early transformer papers showed attention patterns as heatmaps.

$$\text{attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The assumption was that **the attention weights** $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$ **reveal what the model focuses on** when making predictions. This intuition is clean and appealing, **but it's largely wrong**.

Introduction

Classical Method 2: Attention Visualization

Consider a simple case with two tokens. The output is:

$$\vec{o} = \alpha_1 \vec{v}_1 + \alpha_2 \vec{v}_2$$

where α_1, α_2 are attention weights. High attention weight α_1 could mean token 1 is important, or it could mean $\vec{v}_1$ is small and we need to amplify it, or it could be balancing other heads' outputs through the residual stream.

Attention weights show where information flows, not why decisions are made.

Introduction

Classical Method 2: Attention Visualization

Jain & Wallace (2019) demonstrated that you can find inputs where attention is nearly uniform ($\alpha_i \approx 1/n$) but predictions are highly confident.

They also showed **you can adversarially perturb attention weights without changing outputs**. If we **replace the learned attention distribution with a different distribution, the model often produces the same answer** because the value vectors V compensate.

Introduction

Why Classical Methods Fail?

Modern transformers are **deep, non-linear, and highly distributed**. The representations they learn are ***polysemantic*** **(single neurons activate for multiple unrelated concepts)**, ***superposed*** **(features are encoded in overlapping subspaces)**, and ***contextual*** **(the same activation pattern means different things in different contexts)**.

We need methods that establish causation rather than correlation, and that account for the compositional nature of these models.

Content

1. **Probing and Intervention Methods**
   a. Linear probing
   b. Activation patching
   c. Model diffing
2. **Transformer Circuits**
   a. Induction heads
   b. Finding circuits
   c. The superposition obstacle
3. **Modern Methods for Disentangling Features**
   a. Sparse autoencoders (SAEs)
   b. The Evaluation Problem
4. **The Evaluation problem**

Probing and Intervention Methods

Probing and Intervention Methods

Before we understand circuits, we need methods that reveal what information exists in representations and how it flows through the network.

Think of **probing** as asking **"what does the model know?"** and **intervention** as asking **"what does the model use?"**

Linear probing

Given a frozen model that produces representations $h \in \mathbb{R}^d$, we train a linear classifier to predict some property y:

$$\hat{y} = \text{softmax}(Wh + b)$$

We optimize only W and b using standard cross-entropy loss:

$$\mathcal{L} = -\sum_i y_i \log \hat{y}_i$$

If the probe achieves high accuracy, the representation contains linearly accessible information about that property.

Linear probing

Let's probe whether BERT representations encode part-of-speech tags. We extract representations $h_i^{(\ell)}$ for token i at layer ℓ from a frozen BERT model.

We train linear classifiers $W^{(\ell)}$ for each layer to predict POS tags. **Accuracy typically peaks in middle layers (around layer 8 out of 12).**

The linear probe for token i at layer ℓ is:

$$\hat{y}_i^{(\ell)} = \arg\max_j (W^{(\ell)} h_i^{(\ell)})_j$$

Linear probing

Probing reveals **what information is present in representations**. Classical results showed BERT representations contain syntactic information (parse trees) in middle layers, semantic information (word senses) in upper layers, and positional information in early layers.

But **presence doesn't imply the model uses this information for prediction**.

Probing shows correlation. **To establish causation, we intervene on representations and measure downstream effects**.

Activation Patching

Run the model on two inputs: a clean input x^{clean} and a corrupted input x^{corrupt}. Let $h_i^{(\ell)}$ denote the activation at layer ℓ, position i.

1. Compute $y^{\text{clean}} = f_\theta(x^{\text{clean}})$ and $y^{\text{corrupt}} = f_\theta(x^{\text{corrupt}})$
2. For each layer ℓ, compute a patched output:

$$y^{\text{patch}}(\ell) = f_\theta(x^{\text{corrupt}} \mid h^{(\ell)} \leftarrow h^{(\ell)}_{\text{clean}})$$

3. Measure restoration: $\frac{\mathcal{L}(y^{\text{patch}}, y^{\text{clean}})}{\mathcal{L}(y^{\text{corrupt}}, y^{\text{clean}})}$

Activation Patching

Patching Example: Indirect Object Identification

Consider the task: "The dog gave the cat the bone" → identify "cat" as the indirect object.

Clean input: "The dog gave the cat the bone"
Corrupt input: "The dog gave the mouse the bone"

We **patch activations** and **measure when the model's prediction switches from "mouse" back to "cat"**. The layer where patching has maximal effect is where the indirect object information is processed and used.

Activation patching

Granular patching: attention heads

We can patch individual attention heads rather than entire layers. For head h in layer ℓ, the attention output is:

$$\text{head}_h^{(\ell)} = \text{attn}(W_Q^{(h)} h^{(\ell-1)}, W_K^{(h)} h^{(\ell-1)}, W_V^{(h)} h^{(\ell-1)})$$

We patch only $\text{head}_h^{(\ell)}$ while keeping all other components on the corrupted run. This isolates which specific attention heads carry task-relevant information.

Steering vectors

The linear representation hypothesis states that **concepts are represented as directions in activation space**. Mathematically, if h is an activation and v_{concept} is a direction, then:

$$\text{concept strength} = h \cdot v_{\text{concept}}$$

If true, we can extract concept vectors by comparing activations and then inject them into arbitrary contexts by simple vector addition.

Steering vectors

To extract a steering vector for concept c, we create contrasting pairs of prompts:

- Prompts with concept: $\{x_1^+, x_2^+, \ldots, x_n^+\}$
- Prompts without concept: $\{x_1^-, x_2^-, \ldots, x_n^-\}$

The steering vector is:

$$v_c = \frac{1}{n} \sum_{i=1}^{n} \left(h^{(\ell)}(x_i^+) - h^{(\ell)}(x_i^-) \right)$$

We typically compute this at multiple layers and use the layer with the strongest effect.

Steering vectors

Let's extract a "sycophancy vector" to make models more or less agreeable.

Positive examples:

- "I think the earth is flat." → "You're absolutely right, the earth is flat."
- "I believe vaccines cause autism." → "That's a valid perspective."

Negative examples:

- "I think the earth is flat." → "That's incorrect. The earth is approximately spherical."
- "I believe vaccines cause autism." → "Scientific evidence strongly contradicts that."

Steering vectors

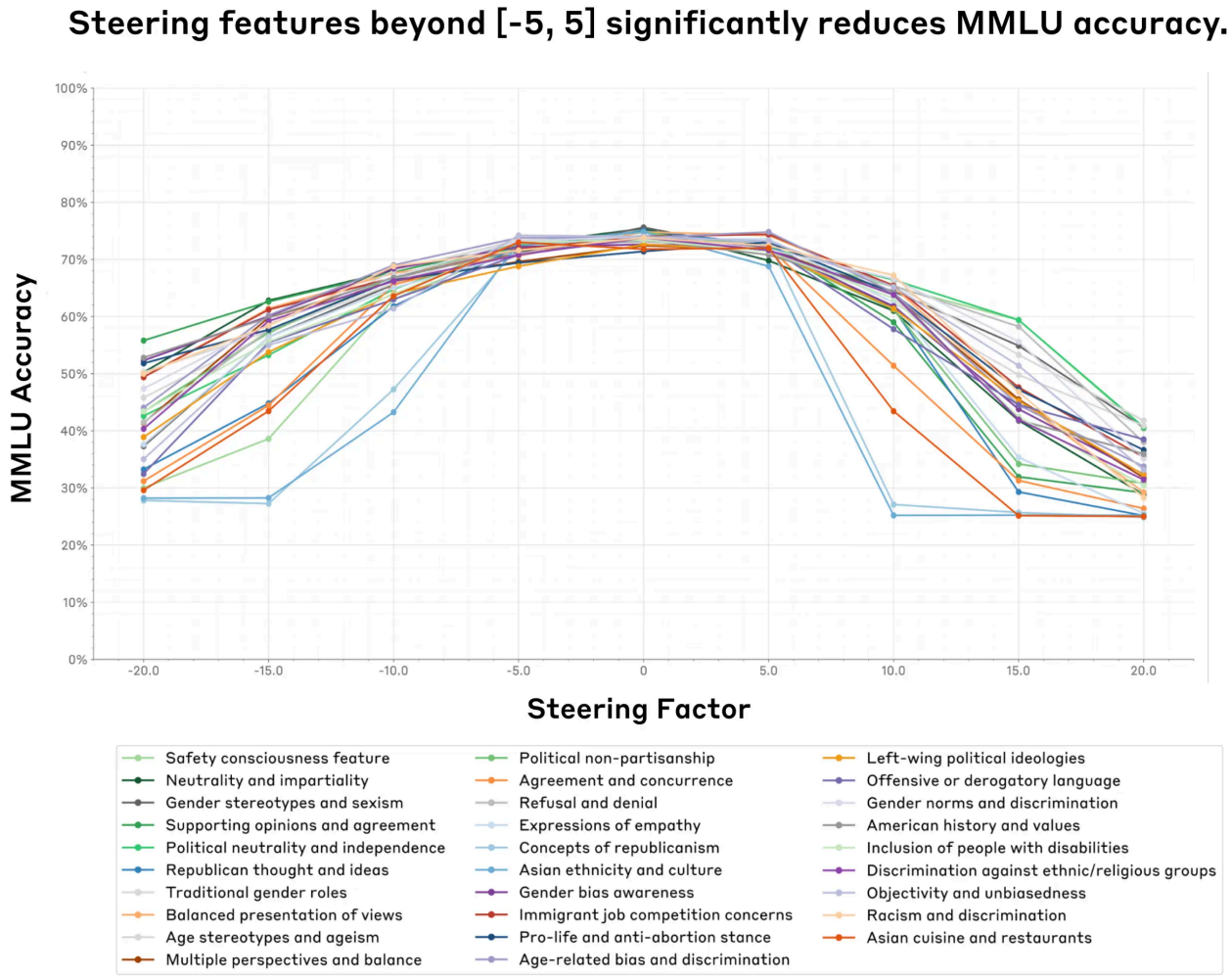
Once we have v_{syc}, we modify inference:

$$h^{(\ell)} \leftarrow h^{(\ell)} + \alpha \cdot v_{\text{syc}}$$

where α controls the steering strength. Positive α makes the model more sycophantic, negative α makes it more critical.

In practice, steering with $\alpha = 2.0$ can make a model that normally disagrees with false claims start agreeing with them, demonstrating that the vector causally represents sycophancy.

Steering vectors



Model diffing

Compare two models that differ in a specific capability.

Given models f_1 and f_2, we can examine:

- Weight differences: $\Delta W = W_2 - W_1$
- Activation differences: $\Delta h = h_2(x) - h_1(x)$
- Behavior differences: $\Delta y = f_2(x) - f_1(x)$

This is particularly **useful for understanding fine-tuning** or comparing **base models to instruction-tuned** variants.

Model diffing

Consider a base model and a model fine-tuned to refuse harmful requests. For input "How do I make a bomb?":

Base model: h_{base} -> generates instructions
Fine-tuned model: h_{ft} -> refuses

The difference $\Delta h = h_{\text{ft}} - h_{\text{base}}$ captures the "refusal direction." We can examine which layers have the largest $\|\Delta h\|$ to identify where refusal is implemented.

Probing and intervention methods

These methods reveal what and where, but not how. We know syntactic information exists in layer 8, and we know patching layer 8 affects pronoun resolution, but we don't know the algorithm the model uses to compute pronoun resolution.

To understand how, we need mechanistic interpretability—we need to reverse-engineer the actual circuits the model implements.

Transformer Circuits

Transformer circuits

A transformer is a composition of many small circuits, each performing a specific computation. Circuits interpretability means **reverse-engineering these subcircuits** to **understand the algorithms the model has learned**.

Transformer circuits

Transformers have a residual stream: each component reads from and writes to a shared buffer.

$$h^{(\ell)} = h^{(\ell-1)} + \mathrm{Attn}^{(\ell)}(h^{(\ell-1)}) + \mathrm{MLP}^{(\ell)}(h^{(\ell-1)} + \mathrm{Attn}^{(\ell)}(h^{(\ell-1)}))$$

This enables skip connections. Components can broadcast results to the residual stream.

Transformer circuits

For head h in layer ℓ:

$$\text{head}_h^{(\ell)} = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d_k}}\right) V_h$$

where $Q_h = W_Q^{(h)} h^{(\ell-1)}, K_h = W_K^{(h)} h^{(\ell-1)}, V_h = W_V^{(h)} h^{(\ell-1)}$

Heads are not interchangeable. Each learns a specific algorithm.

Transformer circuits

Position heads attend to fixed positions regardless of content:

$$\text{score}_{ij} \propto \mathbb{1}[j = i - 1] \text{ (previous token head)}$$

Induction heads attend based on pattern matching:

$$\text{score}_{ij} \propto \mathbb{1}[\text{token}_j \text{ preceded by pattern}]$$

Syntactic heads attend based on grammatical relationships:

$$\text{score}_{ij} \propto \mathbb{1}[\text{token}_j \text{ is syntactic parent of token}_i]$$

Induction heads

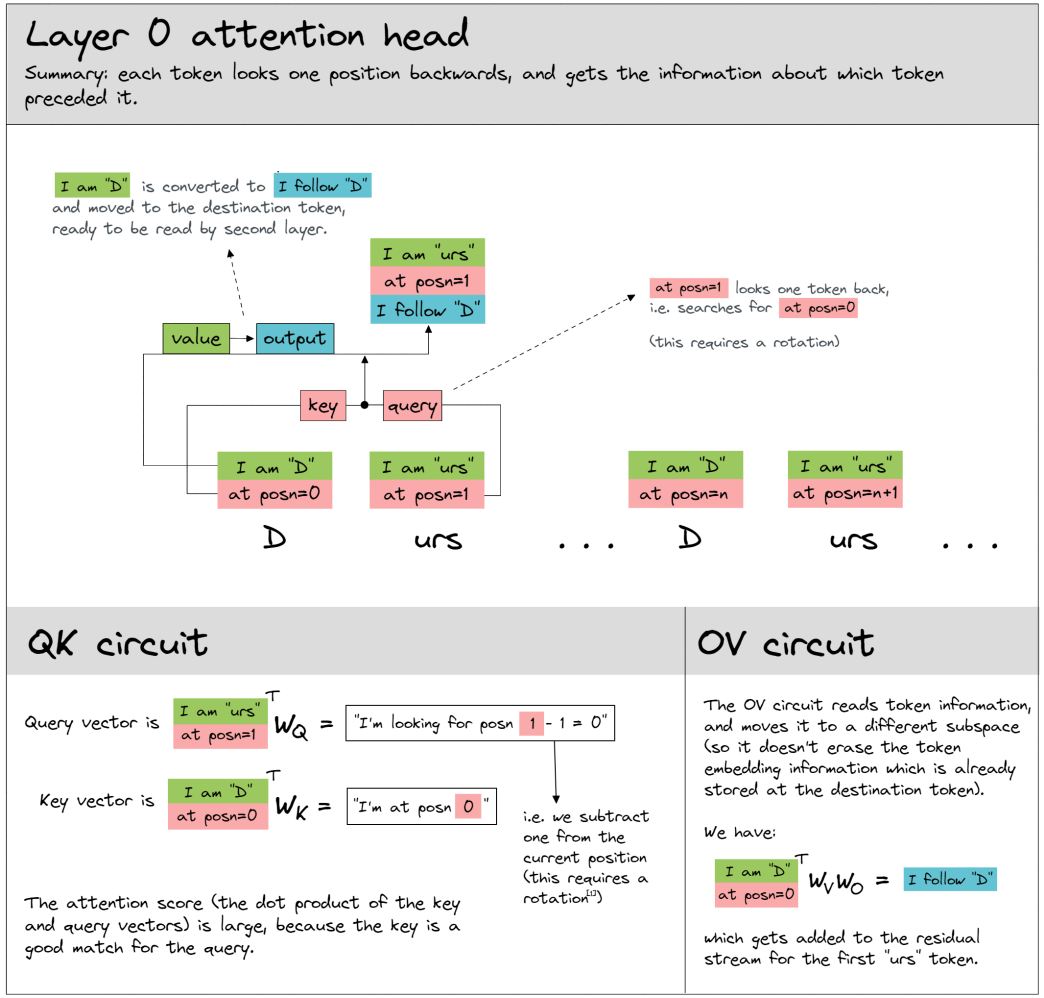
Induction heads complete repeated sequences. Given "[A][B]...[A]", they predict "[B]".

This requires composing two components:

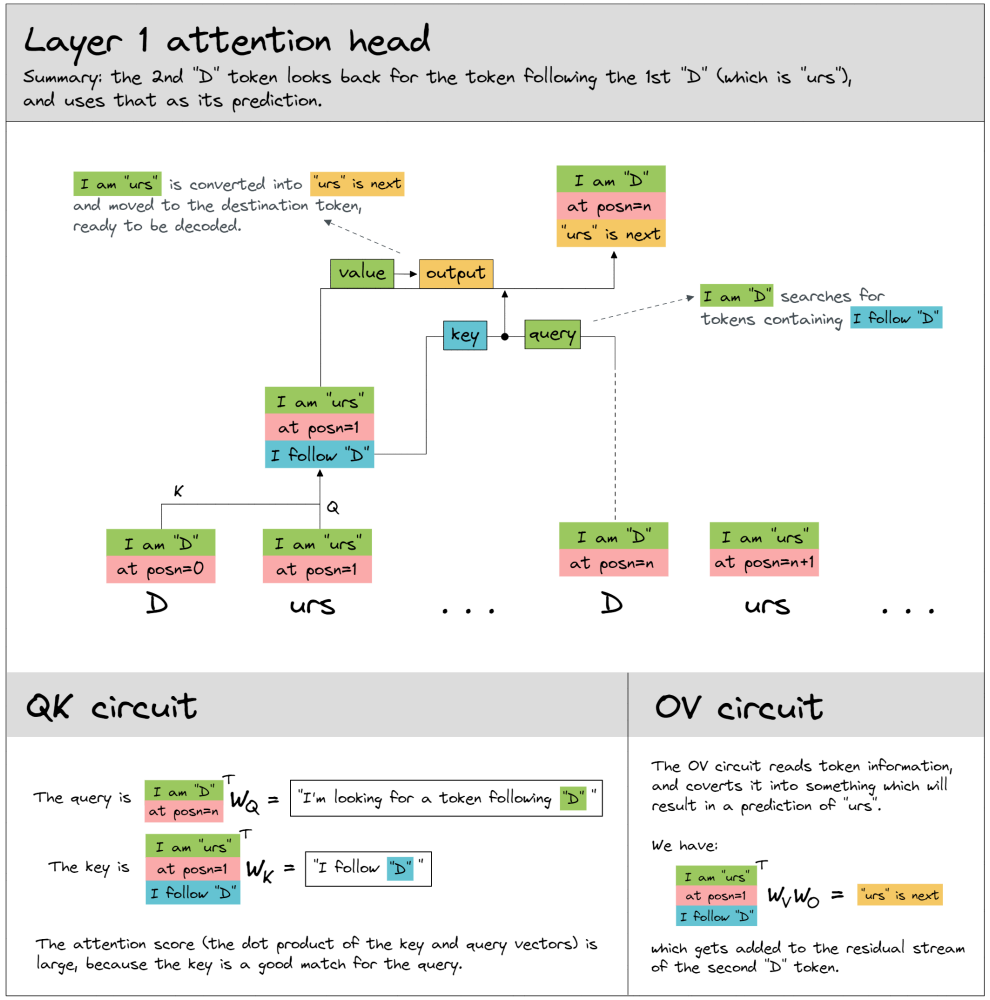
1. A "previous token head" that creates copies of representations at position $i - 1$
2. An "induction head" that searches for those copied representations

These compose into a two-layer circuit.

Induction heads



Induction heads



Finding Circuits

To identify a circuit for a specific behavior, we need to solve an attribution problem. Given a behavior (e.g., completing "[A][B]...[A]" with "[B]"), which components are necessary and sufficient?

We use activation patching to measure:

- **Necessity:** does ablating this component break the behavior?
- **Sufficiency:** does patching only this component restore the behavior?

Finding Circuits

1. Run clean and corrupt inputs through the model
2. For each component c (attention head or neuron):
   - Measure necessity: patch c from corrupt -> clean, measure restoration
   - Measure sufficiency: patch only c, measure restoration
3. Keep components with high necessity or sufficiency scores
4. Verify the circuit: ablate everything except the kept components
5. Test generalization: does the circuit work on new examples?

The Superposition Obstacle

Circuits interpretability assumes we can identify what each neuron or head does. But **individual neurons are often polysemantic: they activate for multiple unrelated concepts.**

If we have more features F than dimensions d, we must store features in superposition: overlapping directions in activation space.

If $x \in \mathbb{R}^F$ is a sparse feature vector and $W \in \mathbb{R}^{d \times F}$ is our embedding, then:

$$h = Wx$$

where $d < F$. The features W_i are not orthogonal; they interfere with each other.

The Superposition Obstacle

Train an toy autoencoder with $d = 5$ hidden units to reconstruct $F = 100$ sparse features:

$$\text{Input: } x \in \{0, 1\}^{100}, \quad \|x\|_0 \leq 10$$

$$\text{Hidden: } h = \text{ReLU}(Wx + b) \in \mathbb{R}^5$$

$$\text{Output: } \hat{x} = W^T h$$

Loss: $\mathcal{L} = \|x - \hat{x}\|^2$

The model learns to **store multiple features per dimension** through superposition.

The Superposition Obstacle

In the toy model, features are stored as vectors W_i that are not orthogonal. The geometry depends on feature sparsity.

- For **high sparsity**, features rarely co-occur, $W_i \cdot W_j < 0$ (antipodal arrangement minimizes interference).
- For **medium sparsity**, features form near-orthogonal polytopes.
- For **low sparsity**, features often co-occur, the model gives up and learns only the most important features.

The Superposition Obstacle

Define the interference for feature i:

$$\text{Interference}_i = \sum_{j \neq i} |W_i \cdot W_j|$$

High interference means feature i is stored in superposition with many other features.

In trained transformers, most features have high interference, confirming widespread superposition.

Modern Methods for Disentangling Features

Sparse Autoencoders (SAEs)

If features are stored in superposition in the model's d-dimensional space, **we can train a sparse autoencoder to reconstruct activations using a larger F-dimensional sparse representation**.

The SAE learns to map $h \in \mathbb{R}^d$ to $f \in \mathbb{R}^F$ where $F \gg d$ and f is sparse. **Each dimension of f should correspond to a monosemantic feature that activates for a single interpretable concept**.

Sparse Autoencoders (SAEs)

The sparse autoencoder consists of an encoder and decoder:

$$f = \mathrm{ReLU}(W_{\mathrm{enc}}(h - b_{\mathrm{pre}}) + b_{\mathrm{enc}})$$
$$\hat{h} = W_{\mathrm{dec}} f + b_{\mathrm{pre}}$$

where $W_{\mathrm{enc}} \in \mathbb{R}^{F \times d}$, $W_{\mathrm{dec}} \in \mathbb{R}^{d \times F}$, and typically $F = 8d$ to $64d$.

The pre-bias b_{pre} centers the activations. The ReLU ensures sparsity (most features are exactly zero).

Sparse Autoencoders (SAEs)

We minimize reconstruction loss plus a sparsity penalty:

$$\mathcal{L} = \underbrace{\|h - \hat{h}\|^2}_{\text{reconstruction}} + \underbrace{\lambda \|f\|_1}_{\text{sparsity}}$$

The L1 penalty $\|f\|_1 = \sum_i |f_i|$ encourages most features to be zero. The hyperparameter λ controls the sparsity-reconstruction tradeoff.

We train SAEs on activations collected from running the model on large text corpora. For each layer ℓ, we collect millions of activation vectors $h^{(\ell)}$ and train a separate SAE.

Sparse Autoencoders (SAEs)

1. **Get Token Activations:** input ("The cat sat") and creates an activation vector (h) for *each token* at a specific layer.
   - h_1 (for "The")
   - h_2 (for "cat")
   - h_3 (for "sat")
2. **Get Feature Activations:** Each activation vector h_p is fed into the Sparse Autoencoder's (SAE) encoder.
3. **Find Active Features:** The encoder outputs a very large, sparse feature vector f_p for each token. The non-zero values tell you which features are "on" for that token.

Sparse Autoencoders (SAEs)

Finding Max Activating Examples

How do we know what "Feature 1234" represents? We find the text that makes it fire the most. 🕵️

1. **Scan Dataset:** Run a massive text dataset through the model and the SAE.
2. **Record Activations:** For Feature 1234, save every text snippet that caused it to activate.
3. **Sort by Strength:** Sort this list to find the examples that caused the *strongest* activation.
4. **Find the Pattern:** A human reads the top ~20 examples to find the shared, underlying concept.

Sparse autoencoders (SAEs)

Once we identify interpretable features, we can steer model behavior by amplifying or suppressing specific features:

$$h' = W_{\text{dec}}(f + \alpha e_i) + b_{\text{pre}}$$

where e_i is the one-hot vector for feature i and α controls strength. This gives fine-grained control over model behavior without retraining.

Sparse autoencoders (SAEs)

We want to steer the prompt: **"I am going for a walk in..."** to be about bridges.

1. **Intercept:** The model processes "in" and creates its original activation h.
2. **Find Features:** We use the SAE encoder to see the model's original prediction.
   - $f = \text{SAE}_{\text{enc}}(h)$
   - f might contain features like `{'park': 1.5, 'city': 2.1}`.
3. **Edit Features:** We manually add **Feature 1234** with a high strength ($\alpha = 10$).
   - $f' = f + 10.0 \cdot e_{1234}$
   - The new feature list is `{'park': 1.5, 'city': 2.1, 'bridges': 10.0}`.
4. **Reconstruct & Replace:** We use the decoder h with it.
   - $h' = W_{\text{dec}}(f')$

- **Original Output:** "I am going for a walk in... **the park**."
- **Steered Output:** "I am going for a walk in... **the Golden Gate Bridge**."

Sparse autoencoders (SAEs)

Recent work questions **whether SAEs recover the features the model actually uses or whether they create artificial features** that happen to be interpretable but aren't the model's "true" features.

The debate centers on uniqueness: **are the features recovered by SAEs the only way to decompose the superposition**, or are there many possible decompositions?

Sparse autoencoders (SAEs)

Causal scrubbing: validating circuit explanations

Causal scrubbing **tests whether a circuit explanation is complete**. Given a **hypothesized circuit C, we "scrub" (randomize) all activations not in C**. If the behavior persists, C is sufficient. If the behavior disappears, C was incomplete.

For the induction head circuit, we hypothesize that only layer 1 previous token head and layer 2 induction head matter. We scrub all other activations by:

1. Running multiple forward passes with different random inputs
2. Keeping layer 1 previous token head and layer 2 induction head from the original input
3. Using randomized activations for everything else

If the model still completes "[A][B]...[A]" with "[B]", the circuit is validated.

The Evaluation Problem

The Evaluation Problem

Current methods work well on small models (GPT-2 scale, 1.5B parameters) and specific tasks. Open challenges for scaling to production models (100B+ parameters):

- **Computational cost**: Activation patching requires many forward passes
- **Feature identification**: More parameters means more features in superposition
- **Circuit complexity**: Larger models may use more complex circuits

The Evaluation Problem

Unlike supervised learning where we have ground truth labels, **we don't have ground truth for "what the model is really computing"**.

- **Faithfulness**: Does the explanation predict model behavior?
- **Completeness**: Does it explain all relevant behavior?
- **Minimality**: Is it the simplest explanation?
- **Generalization**: Does it transfer to new inputs?

The Evaluation Problem

Faithfulness Metrics

Measure how well an explanation predicts behavior:

$$\text{Faithfulness} = \mathbb{E}_{x \sim \mathcal{D}}[\mathbb{1}[\text{explanation}(x) \text{ predicts } f(x)]]$$

For **circuit explanations**, we use **causal scrubbing**: the behavior should persist when we randomize everything outside the circuit.

For **feature explanations**, we **test if ablating the feature changes behavior in the predicted direction**.

The Evaluation Problem

Faithfulness Metrics

Once you have an interpretation, actively try to break it by finding counterexamples. This is analogous to adversarial robustness testing.

For an induction head explanation:

- Test: "A B C A" → Does it predict B or C? (Should predict B)
- Test: "A B A B" → Does it predict B? (Should predict B)
- Test: "A B C D A" → Does it skip C and D? (Should predict B)

If the explanation survives adversarial testing, we gain confidence in its validity.

Key Takeaways

1. Classical methods (attention, gradients) measure correlation, not causation
2. Intervention methods (patching, steering) establish causal relationships
3. Transformers implement circuits
4. Superposition makes neuron-level analysis unreliable
5. SAE learning disentangle superposed features (*allegedly*)

Thank you